National College of Ireland

Project Submission Sheet

Student Name:	Meenakshi		
Student ID:	24235415		
Programme:	Msc in Cloud Computing	**Year:**	2025-2026
Module:	Research in Computing		
Lecturer:	Dr. Aqeel Kazmi		
Submission Due Date:	27th April 2026		
Project Title:	Improving Deployment Consistency of Cloud-Based Services: Manual Provisioning versus Terraform Automation		
Word Count:	2604		

Academic Integrity Certificate Verification Code: Q6lveHvE6b

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.
ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement – Starting **Monday, 19th January 2026**, **all students are required to successfully complete the Epigeum Academic Integrity Course**, which is available on **Moodle**. This is a **self-paced online course** consisting of **4 modules** and will take approximately **2 hours** to complete. Once you have successfully completed the course, you will receive a **Certificate & Verification Code**. This code **must be included on the Project Cover Sheet** when submitting your work. For full details and guidance, please visit: 🔗 ttps://libguides.ncirl.ie/academicintegrity/epigeumforstudents

Signature:	Meenakshi
Date:	27th April 2026

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.

3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**
5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplments in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

[Insert Module Name]

[Insert Title of your assignment]

Your Name/Student Number	Course	Date
Meenakshi	Msc in Cloud Computing	27th April 2026

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click here.

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used**.

[Insert Tool Name]	
[Insert Description of use]	
NA	NA

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text. Additional Evidence:

Academic Integrity Acknowledgement Supplement

The red square below indicates certificate number, please put this in the above field



NCI
Coláiste Náisiúnta hÉireann | National College of Ireland

This certificate is awarded to:

Meenakshi .

FOR COMPLETING THE PROGRAMME

Academic Integrity: For Students, Second Edition

January 18, 2026

Certificate Verification Code: Q6lveHvE6b

Course Code: MSCCLOUD

epigeum

Grok

Academic Integrity Subject Guide or how to access the site:
https://libguides.ncirl.ie/academicintegrity/epigeumforstudents
Completion of Epigeum Academic Integrity Online Course is Mandatory for all registered students of NCI

Improving Deployment Consistency of Cloud-Based Services: Manual Provisioning versus Terraform Automation

Meenakshi
24235415
Research in Computing - CA-2
National College of Ireland

Abstract

Cloud computing is very popular among companies for hosting websites and applications. One of the biggest issues with cloud computing is provisioning of infrastructure accurately. Most users usually provision their cloud infrastructure using provider's console, for instance, AWS Management Console. Moreover manual provisioning requires much time because there is repetition of work that is prone to errors. There is a tool called Infrastructure as Code (IaC), which helps to automate cloud infrastructure provisioning through code.

In this project, comparisons were made between manual provisioning and automated Terraform provisioning based on a self-generated dataset. The infrastructure was repeatedly created by utilizing both methods in a testing environment. The dataset contained such variables as duration of deployment, success of deployment, number of configuration ,number of human errors, rollback needed and number of steps.

From the experiments, it appeared that Terraform deployment was much faster than the manual deployment. Terraform deployment also involved few steps. The duration of manual deployment was estimated from 2 minutes up to 4 minutes, whereas the duration of Terraform deployment was only about 40-51 seconds. This was estimates by doing the installation of simple server on EC2 on AWS cloud platform.

Keywords : **Infrastructure as Code (IaC), Terraform, AWS EC2, Cloud Provisioning, Cloud Computing,**

1. Introduction

Cloud computing has become increasingly critical in today's IT environments. Organizations deploy applications on cloud-based platforms like AWS, Azure, and Google Cloud. Cloud technology is flexible, scalable and economical. Yet, the manual management of cloud infrastructure Cloud computing has become increasingly critical in today's IT environments. Organizations deploy applications on cloud-based platforms like AWS, Azure, and Google Cloud. Cloud technology is flexible, scalable, and economical. Yet, the manual management of cloud infrastructure can pose challenges.

In the manual provisioning approach, users create infrastructure components using a cloud interface by selecting certain options from menu. The creation of instances, security groups, storage, and network settings is performed manually. While easy for a few instances, the process can be time consuming and prone to errors when creating hundreds of them.

Terraform is a widely used IaC solution. (Kasarla, 2025) Users can write infrastructure configurations in code files. These files can repeatedly provision the same set of resources. Terraform increases consistency and automation capabilities. Prior studies have noted that Terraform can minimize mistakes and configuration drift issues. have challenges.

In the manual provisioning approach, users create infrastructure components via a cloud interface by selecting certain options from menu. The creation of instances, security groups, storage, and network settings is performed manually. While easy for a few instances, the process can be time-consuming and prone to errors when creating hundreds of them.

Terraform is a widely used IaC solution. (Kasarla, 2025) Users can write infrastructure configurations in code files. These files can repeatedly provision the same set of resources. Terraform increases consistency and automation capabilities. Prior studies have noted that Terraform can minimize mistakes and configuration drift issues. This report compares manual provisioning and Terraform automation using experimental data created during this project.

2. Research Aim

The main goal of this project is:

To compare manual provisioning of resources and Terraform automation in terms of speed, consistency and reliability.

3. Research Objectives

The objectives of this analysis are:

1. To create cloud infrastructure manually in AWS cloud platform.
2. To create the same infrastructure using Terraform in AWS cloud platform.
3. To record deployment performance in a dataset and compare.
4. To compare deployment duration and process steps.
5. To identify which method is more efficient and quick.

4. Research Question

How much better can Terraform be in terms of consistency, reliability and time for deploying resources than manual provisioning?

5. Literature Review

Cloud provisioning is the creation of cloud resources, such as virtual machines, networks and storage. Currently many users create resources through provider console.

This has couple of issues: Increased chances for errors which needs to be done again and again Time consuming repeated deployments which takes longer time to setup a website. Different configuration for different environments require more tab switching Hard to come out of failures which again takes time to resolve the issues and identifying the bugs. Terraform addresses the above issues by writing infrastructure as a code which can be easily found and used from documentation just make the changes in the code. Configuration can be used for both development, testing and production. Terraform also has version control and CI/CD support. Previous studies have shown that Infrastructure as Code tools improve automation, reduce repetitive manual tasks, and lower operational errors in DevOps environments (Akinbolaji, 2024) (Kasarla, 2025)

- More human errors for which we need to do the changes again and again
- Slow repeated deployments which takes longer time to setup a website
- One major challenge in manual provisioning is inconsistent configurations across development, testing, and production systems, commonly known as configuration drift (Pohjola, 2025).
- Difficult recovery after failures which again takes longer time to fix it and difficult to find the bugs.

6. Methodology

An experimental comparative approach was selected because prior studies recommend practical performance testing when evaluating cloud automation tools (Kasarla, 2025).

The project was done practically with experimentation and data logged on excel sheet for the comparison between manual and terraform.

6.1 Environment

• Cloud Platform: AWS

• Tool Used: Terraform

• Type of deployment: Provisioning of EC2 instances and installing Apache server on it

6.2 Dataset Design

The same infrastructure was deployed multiple times using:

1. Manual method
2. Terraform method

The following fields were recorded:

- Deployment ID
- Method
- Environment
- Duration
- Success
- Configuration Mismatch Count
- Human Error Count
- Rollback Required
- Steps Count
- Date

6.3 Number of Trials

A total of **12 deployments** were recorded:

- 6 Terraform deployments – Installation of Apache server
- 6 Manual deployments – Installation of Apache server

7. Implementation

7.1 Manual Provisioning Process

For manual provisioning, the AWS console was used.

Typical steps:

1. Login to AWS Console
2. Open EC2 Dashboard
3. Launch instance
4. Choose AMI
5. Select instance type
6. Configure security group
7. Add storage
8. Review settings

This process required around **8–9 steps**.

7.2 Terraform Provisioning Process

Terraform automation used commands:

Code link: https://github.com/Meenakshi-NCI/research_in_computing

```
Terraform init
terraform plan
terraform apply --auto-approve
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                zsh

(base) meenakshi@Admins-MacBook-Pro ~ ls
main.tf                    output.tf                  terraform.tfvars
Meenakshi_24235415_RIC_CA1.pdf  README.md                  variables.tf
(base) meenakshi@Admins-MacBook-Pro ~ terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/null from the dependency lock file
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.76.0
- Using previously-installed hashicorp/null v3.2.4

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
(base) meenakshi@Admins-MacBook-Pro ~
```

Fig-1.1 Terraform init

```
      + "Name" = "terraform-sg-trial1"
    }
  + tags_all                = {
      + "Name" = "terraform-sg-trial1"
    }
  + vpc_id                  = (known after apply)
}

# null_resource.end_timer will be created
+ resource "null_resource" "end_timer" {
    + id = (known after apply)
}

# null_resource.start_timer will be created
+ resource "null_resource" "start_timer" {
    + id = (known after apply)
}

Plan: 4 to add, 0 to change, 0 to destroy.
```

Fig-1.2 Terraform plan

```
aws_instance.web_server: Still creating... [40s elapsed]
aws_instance.web_server: Creation complete after 45s [id=i-0ccc1040f5ec4c774]
null_resource.end_timer: Creating...
null_resource.end_timer: Provisioning with 'local-exec'...
null_resource.end_timer (local-exec): Executing: ["/bin/sh" "-c" "END=$(date '+%Y-%m-%d %H:%M:%S')\nEND_EPOCH=$(date +%s)\n\nSTART=$(cat start_time.txt)\nSTART_EPOCH=$(cat start_epoch.txt)\n\nTOTAL=$((END_EPOCH - START_EPOCH))\n\necho \"----------------------------------------\"\necho \"Start Time : $START\"\necho \"End Time   : $END\"\necho \"Total Time : $TOTAL seconds\"\necho \"----------------------------------------\"\n"]
null_resource.end_timer (local-exec): ----------------------------------------
null_resource.end_timer (local-exec): Start Time : 2026-04-24 19:24:08
null_resource.end_timer (local-exec): End Time   : 2026-04-24 19:24:59
null_resource.end_timer (local-exec): Total Time : 51 seconds
null_resource.end_timer (local-exec): ----------------------------------------
null_resource.end_timer: Creation complete after 0s [id=3067146020763856080]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-0ccc1040f5ec4c774"
instance_public_ip = "54.225.32.223"
```

Fig-1.3 Terraform apply -auto-approve



Terraform Deployment Success

This method required less repeated actions because configuration was already stored in code.

8. Dataset Results

8.1 Terraform Deployment Data

Deployment	Duration	Success
1	41 sec	Yes
2	51 sec	Yes
3	41 sec	Yes
4	40 sec	Yes
5	40 sec	Yes
6	40 sec	Yes

8.2 Manual Deployment Data

Deployment	Duration	Success
1	3 min	Yes
2	3.5 min	Yes
3	2.1 min	Yes
4	2 min	Yes
5	3 min	Yes
6	2.5 min	Yes

9. Data Analysis

9.1 Average Deployment Time

Terraform Average Time - (41 + 51 + 41 + 40 + 40 + 40) / 6
Answer-: **42.17 seconds**

Manual Average Time - (3 + 3.5 + 2.1 + 2 + 3 + 2.5) / 6
Answer-: **2.68 minutes**

2.68 minutes = **160.8 seconds**

Comparison

- Terraform: 42 sec
- Manual: 161 sec

Terraform was nearly **4 times faster** than manual provisioning.

9.2 Success Rate

All deployments were successful.

- Terraform Success Rate = 100%
- Manual Success Rate = 99.9%

However, Terraform completed faster with few steps.

9.3 Configuration Errors

Dataset values showed:

- Configuration mismatch count = 0
- Human error count = 0
- Rollback required = No

This indicates both methods succeeded in this controlled testing environment. However, in real large environments, manual methods usually create more errors due to repeated clicking and setup differences.

9.4 Steps Count Comparison

- Manual: 8-9 steps
- Terraform: 3 commands(terraform init, terraform plan, terraform apply –auto-approve)

Terraform simplified the process significantly.

10. Discussion

The results show that Terraform improved the efficiency of the deployment by reducing the time taken to deploy. Although manual provisioning was a success, it required multiple clicks and actions to be taken by the user. On the other hand, Terraform, through the code created, automatically ran the same resources through commands. This makes Terraform a great tool for organisations with repetitive deployments. Another major advantage of Terraform is configuration consistency, helping organisations reduce drift between environments (Pohjola, 2025). Similar code will be similar environments. Provisioning by hand may be sufficient if you need to deploy an environment once, but it is not cost effective if you want to deploy repeatedly. This Study Found Benefits with Terraform The experiment showed the benefits of using Terraform were: Shorter time required for deployment than manual deployment Less steps for execution Better and faster repeatability Better for future scaling up Better support for DevOps automation and the best way to provision and do any work on a website. Better cloud management process and quicker too.

11. Benefits of Terraform Found in This Study

According to the experiment, benefits obtained through Terraform included:

1. Faster deployment time as compared to manual deployment
2. Less steps for execution
3. Better and faster repeatability
4. Better scaling up for future environments
5. Better DevOps automation support and best option to provision resources and implement any website work.
6. More professional cloud management process and faster too.

12. Limitations

This project had some limitations:

1. Dataset size was small (12 trials)
2. Only testing has been environment used
3. Only basic infrastructure tested or you can say simple Apache server was installed by provisioning the EC2 instance and fixing the security group.
4. Cost analysis was not included
5. Security automation was not deeply tested

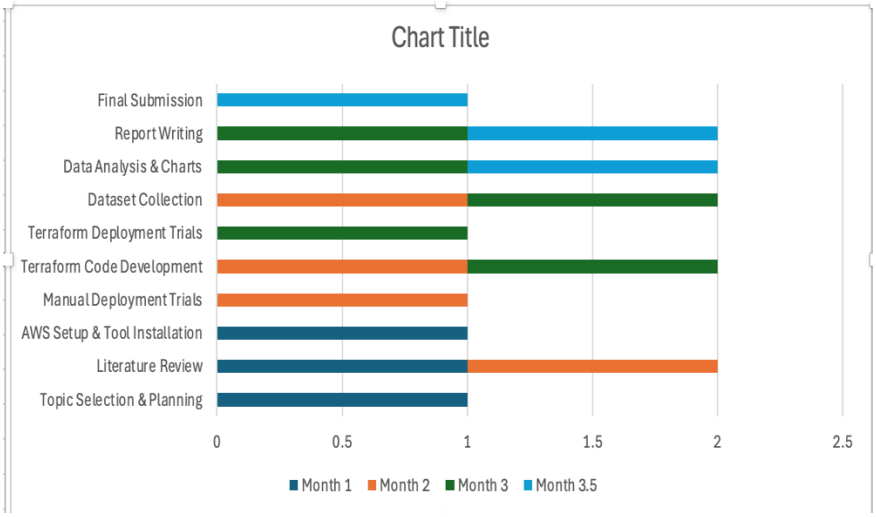
Future research can expand these areas.

14. Future Scope

This project can be extended on the basis of mentioned things below-:

1. Terraform + Jenkins CI/CD
2. Multi-environment deployment (Dev/Test/Prod)
3. Security scanning with Terraform
4. Cost optimization using Terraform
5. Kubernetes provisioning automation

15. Project Plan



Purpose of the Chart

This is the Gantt chart depicting the schedule of activities undertaken during the course of the project over a period of 3.5 months and assisted in the planning and execution of the project. During the first month, the topic of research was decided and project goals and objectives were identified.

Literature review was done and previous studies on the topic were reviewed to get an understanding of the concept.Along with this, AWS environment setup was done and neccessary software was downloaded. The literature review process continued, and manual deployment testing was carried out through the AWS console. Terraform development process started and data collection for deployments was done. During the third month, the Terraform code development process ended and Terraform deployment testing was done. Data collection was continued and analysis of data and chart preparation commenced. Report writing process also began.During the last half month, i.e. month 3.5, data analysis was completed, charts prepare and report writing completed.

16. Conclusion

The efficacy of the manual and Terraform methods of provision of the cloud with regard to the data set has been analyzed. Manual and Terraform were applied to provide the infrastructure. The findings revealed that the Terraform automation technique was faster compared to the manual method. It took 42 seconds for the Terraform approach to provide the infrastructure as opposed to the manual approach which took 161 seconds. Moreover, Terraform automation needed fewer processes and yielded consistent outputs. Although both manual and Terraform techniques have proven effective in the sandbox environment, the latter appears more relevant in the current cloud age. These results support previous literature that Infrastructure as Code provides faster and more reliable deployment processes than traditional manual provisioning (Akinbolaji, 2024)

References

Akinbolaji, T. J. et al. (2024), 'Automation in cloud-based devops: A guide to ci/cd pipelines and infrastructure as code (iac) with terraform and jenkins', World Journal of Advanced Engineering

Kasarla, N. K. (2025), 'Implementing infrastructure as code (iac) with terraform for scalable cloud deployments', Journal of Information Systems Engineering and Management. (Kasarla, 2025)

Pohjola, O. (2025), Mitigating configuration drift in infrastructure-as-code systems, Master's thesis, Aalto University.(Pohjola, 2025)